Distributed Availability Groups (DAG)

Architecture and Real-Time Scenarios in SQL Server

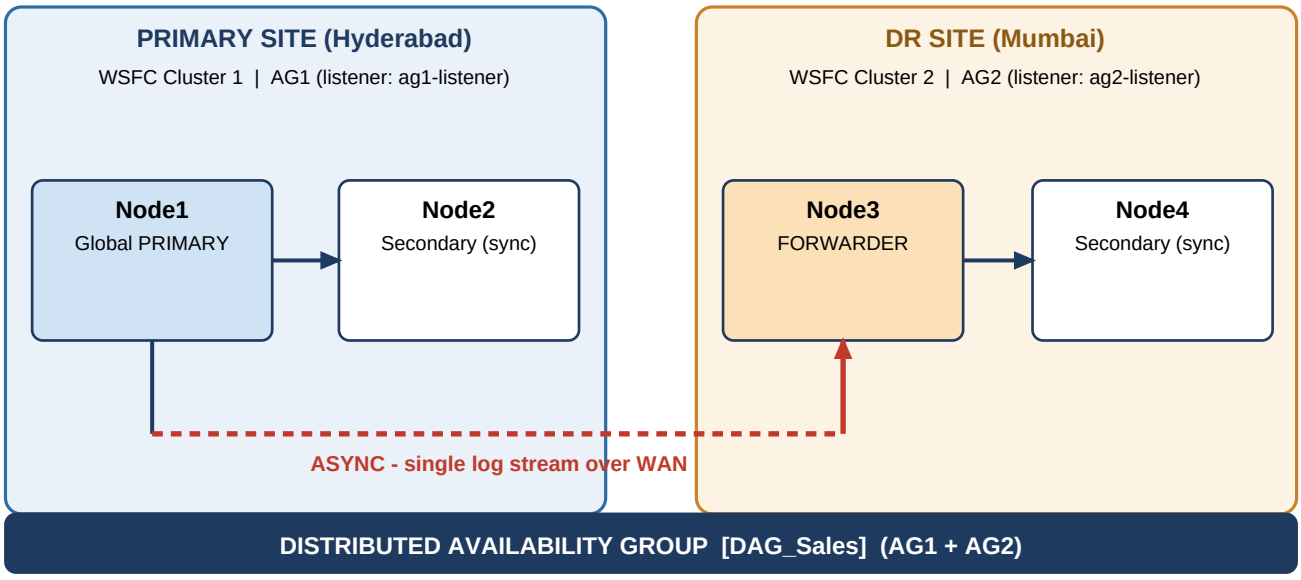
This document explains Distributed Availability Groups, the Always On feature that links two independent Availability Groups. If you meant Directed Acyclic Graphs (data pipeline workflows such as SSIS or ADF dependencies), that is a different topic.

1. What is a DAG?

A **Distributed Availability Group** is a special availability group that spans **two separate Availability Groups (AGs)**. Each AG can sit on its own Windows Server Failover Cluster (WSFC), in a different data center, region, domain, or even a different OS.

A normal AG is limited to one WSFC. A DAG removes that limit by linking two independent AGs, so you do not need a stretched cluster.

2. Architecture



Key components

Component	Role
Underlying AG1 / AG2	Normal AGs, each with its own replicas and listener, managed by its own cluster.
Global Primary	The primary replica of AG1. It accepts all read/write traffic.
Forwarder	The primary replica of AG2. It receives the log stream from the global primary and forwards it to its local secondaries.
Database Mirroring Endpoint	TCP port (usually 5022) used for log transport between the AGs.
Listener URL	`TCP://ag1-listener:5022`, the address AG2 uses to reach AG1.
Seeding mode	Automatic seeding, or manual backup/restore, to initialize AG2 databases.

Why the forwarder matters

Only **one log stream** crosses the WAN, from Global Primary to Forwarder. The forwarder then fans out to its own secondaries locally. With 4 DR replicas, that saves 3 WAN streams, which cuts bandwidth and reduces load on the primary.

3. Data flow (step by step)

1. The application commits a transaction on the Global Primary (Node1).
2. The log block is hardened locally, then sent to Node2 (synchronous within AG1).
3. The same log block goes **asynchronously** across the DAG link to the Forwarder (Node3).
4. Node3 hardens it, then sends it to Node4 inside AG2.
5. The redo thread applies the changes on each secondary.

Within an AG the commit mode can be synchronous or asynchronous. **Between AGs in a DAG it is almost always asynchronous**, because synchronous over a WAN adds commit latency to every transaction.

4. Setup (T-SQL outline)

On AG1's primary, create the DAG:

```
CREATE AVAILABILITY GROUP [DAG_Sales]
   WITH (DISTRIBUTED)
   AVAILABILITY GROUP ON
      'AG1' WITH (
         LISTENER_URL = 'tcp://ag1-listener.corp.local:5022',
         AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,
         FAILOVER_MODE = MANUAL,
         SEEDING_MODE = AUTOMATIC),
      'AG2' WITH (
         LISTENER_URL = 'tcp://ag2-listener.dr.local:5022',
         AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,
         FAILOVER_MODE = MANUAL,
         SEEDING_MODE = AUTOMATIC);
```

On AG2's primary, join it:

```
ALTER AVAILABILITY GROUP [DAG_Sales]
   JOIN AVAILABILITY GROUP ON
      'AG1' WITH (LISTENER_URL = 'tcp://ag1-listener.corp.local:5022',
                  AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,
                  FAILOVER_MODE = MANUAL,
                  SEEDING_MODE = AUTOMATIC),
      'AG2' WITH (LISTENER_URL = 'tcp://ag2-listener.dr.local:5022',
                  AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,
                  FAILOVER_MODE = MANUAL,
                  SEEDING_MODE = AUTOMATIC);
```

Prerequisites

- Enterprise Edition (Basic AGs cannot be part of a DAG).
- Same database names in both AGs, and the databases must be in FULL recovery.
- Network connectivity and firewall rules for the endpoint port.
- Certificate authentication if the AGs are in different domains.
- The secondary AG's SQL Server version must be the same as or higher than the primary's.

5. Failover in a DAG

There is no automatic failover between AGs, only manual. Automatic failover still works *inside* each AG (Node1 to Node2) through WSFC.

Planned failover (zero data loss)

```
-- 1. On Global Primary: switch DAG to synchronous
ALTER AVAILABILITY GROUP [DAG_Sales]
  MODIFY AVAILABILITY GROUP ON
    'AG1' WITH (AVAILABILITY_MODE = SYNCHRONOUS_COMMIT),
    'AG2' WITH (AVAILABILITY_MODE = SYNCHRONOUS_COMMIT);

-- 2. Wait until sys.dm_hadr_database_replica_states shows SYNCHRONIZED
--    and last_hardened_lsn matches on both sides

-- 3. On Global Primary: demote it
ALTER AVAILABILITY GROUP [DAG_Sales] SET (ROLE = SECONDARY);

-- 4. On the Forwarder (AG2 primary): promote
ALTER AVAILABILITY GROUP [DAG_Sales] FORCE_FAILOVER_ALLOW_DATA_LOSS;
-- (no data loss because step 2 ensured sync)

-- 5. Optionally set back to ASYNCHRONOUS_COMMIT
```

Disaster failover (primary site is down): run only step 4. Any unreplicated transactions are lost, so your **RPO equals the async lag at the time of failure**.

6. Real-time scenarios

Scenario 1: Cross-region disaster recovery

An e-commerce company runs a 2-node AG in Hyderabad and a 2-node AG in Mumbai, joined with a DAG. If the Hyderabad data center fails, DBAs run a forced failover to Mumbai and repoint the app through DNS or a global traffic manager. Expected RPO is seconds and RTO is minutes.

Scenario 2: Zero-downtime migration

Migrating from SQL Server 2016 on Windows (on-prem) to SQL Server 2022 on Linux (Azure VM): build the new AG on the new platform, create a DAG between old and new, let it sync, do a planned failover in a short window, then remove the DAG and decommission the old servers. Downtime is roughly 1 to 5 minutes, compared to hours with backup/restore.

Scenario 3: Version upgrade

The DAG lets you move across SQL versions and different WSFCs without in-place upgrades. Because the secondary AG must be at the same or a higher version, the old version always sits on the primary side.

Scenario 4: Read scale-out in a remote region

A reporting team in Mumbai needs near-real-time data. They query the readable secondaries in AG2 (via read-only routing), which keeps heavy reports off the production primary.

Scenario 5: Different domains or no shared cluster

After a merger, Company A (domain A) and Company B (domain B) need replication. A DAG with certificate-based endpoint authentication works without domain trust or a stretched cluster.

Scenario 6: Banking, tiered DR

AG1: 3 nodes, synchronous, primary data center (zero data loss locally). AG2: 3 nodes in a DR region. DAG: asynchronous across regions. This gives local high availability with automatic failover plus regional disaster recovery.

7. Monitoring

```
-- List DAGs
SELECT name, is_distributed FROM sys.availability_groups WHERE is_distributed = 1;

-- Replication health and lag
SELECT ag.name, drs.database_id, DB_NAME(drs.database_id) AS db,
       drs.synchronization_state_desc, drs.synchronization_health_desc,
       drs.log_send_queue_size, drs.log_send_rate,
       drs.redo_queue_size, drs.redo_rate,
       drs.last_hardened_lsn, drs.last_commit_time
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_groups ag ON drs.group_id = ag.group_id;
```

Watch these:

- **log_send_queue_size**: data not yet sent, which is potential data loss
- **redo_queue_size**: data received but not yet applied, which affects RTO
- Network latency and bandwidth on the endpoint port
- Perfmon counters under SQLServer:Availability Replica and SQLServer:Database Replica

8. Limitations and gotchas

- Failover between AGs is **manual only**.
- Two AGs per DAG is the common design. Chaining is possible but complex.
- The DAG itself does not create a listener. The app connects to each AG's own listener, so you need DNS or traffic-manager switching after failover.
- Databases are managed at the AG level. Adding a database requires adding it to both AGs.
- Large log generation over a slow WAN causes growing log_send_queue and log file growth on the primary.
- Use the same SQL logins, SIDs, and agent jobs on both sides, because they are not replicated automatically.
- Test failover regularly. An untested DR plan often fails when it is needed.

9. Best practices

1. Use async commit across regions and sync only inside the local AG.
2. Size the WAN link for peak log generation.
3. Use automatic seeding for large databases only if the network is stable. Otherwise use backup/restore.
4. Sync logins, jobs, and linked servers (for example with dbatools).
5. Set up alerts on queue sizes and health.
6. Document and rehearse the failover runbook.
7. Put the app connection string behind DNS aliases, such as MultiSubnetFailover=True.